

Μεταφραστές

Αναστάσιος-Ορέστης Στέφας 4916

Αμαλία-Γεωργία Μουσελίμη 5074

Περιεχόμενα

Μεταφραστές	1
Εισαγωγή	2
Φάση 1	2
Λεκτική Ανάλυση	2
Συντακτική Ανάλυση	2
Φάση 2	9
Παραγωγή Ενδιάμεσου Κώδικα	9
Πίνακας Συμβόλων	11
Φάση 3	14
Τελικός Κώδικας	14

Εισαγωγή

Για την υλοποίηση της εργασίας του μαθήματος ζητείται η δημιουργία ενός μεταγλωττιστή για μία συγκεκριμένη γλώσσα προγραμματισμού που μας δόθηκε. Η γλώσσα αυτή ονομάζεται cry και αποτελεί μια μικρή γλώσσα προγραμματισμού που δημιουργήθηκε για εκπαιδευτικούς λόγους. Η cry υποστηρίζει συναρτήσεις, μετάδοση παραμέτρων, αναδρομικές κλήσεις, και επιτρέπει φώλιασμα στη δήλωση συναρτήσεων. Ο μεταγλωττιστής υλοποιείται σε γλώσσα python και τα προγράμματα τα οποία καλούμαστε να μεταγλωττίσουμε θα πρέπει να έχουν κατάληξη .cry. Η τελική γλώσσα που παράγεται είναι η γλώσσα assembly του επεξεργαστή RISC-V. Η γλώσσα cry, όπως και κάθε γλώσσα προγραμματισμού, ακολουθεί τη δική της γραμματική, το δικό της αλφάβητο καθώς και μία διαφορετική δομή.

Η εργαστηριακή άσκηση παραδόθηκε σε 3 φάσεις:

1. Λεκτική και Συντακτική Ανάλυση
2. Παραγωγή Ενδιάμεσου Κώδικα και Πίνακα Συμβόλων
3. Τελικός Κώδικας και Δημιουργία Αναφοράς

Φάση 1

Λεκτική Ανάλυση

Ο λεκτικός αναλυτής είναι μία συνάρτηση [lex()] που κάθε φορά καλείται από τον συντακτικό αναλυτή, διαβάζει ένα αρχείο .cry και επιστρέφει μία λεκτική μονάδα. Με τον όρο λεκτική μονάδα αναπαριστούμε οποιοδήποτε σύμβολο ή σύνολο συμβόλων το οποίο ανήκει στο αλφάβητο της γλώσσας cry. Ο λεκτικός αναλυτής διαβάζει το σύμβολο ή το σύνολο συμβόλων και αν αναγνωρίσει την λεκτική μονάδα τότε την επιστρέφει στη συνάρτηση του συντακτικού αναλυτή. Ανάλογα με το πρώτο σύμβολο που διαβάζει, θέτει τον δείκτη state στην εκάστοτε τιμή και στη συνέχεια με έλεγχο στην τιμή state προσπαθεί να αναγνωρίσει σε ποια από τις γνωστές δομές της γλώσσας ανήκει. Σε περίπτωση που δεν αναγνωρίσει την λεκτική μονάδα διακόπτει την μεταγλώττιση και εμφανίζει μήνυμα λάθους. Για την υλοποίηση της συνάρτησης αυτής ορίστηκαν οι λίστες symbols, cond και keywords οι οποίες περιέχουν τα σύμβολα που υπάρχουν στην γλώσσα, τους τελεστές συσχέτισης και τις δεσμευμένες λέξεις αντίστοιχα. Τέλος, δημιουργήθηκαν οι λίστες ID, temp_ID και funcID στις οποίες εντάσσουμε την τιμή του lexUnit όταν δηλώνεται κάποιο όνομα αναλόγως εάν ανήκει στο id, σε κάποιο προσωρινό (temp) id ή είναι το όνομα μιας συνάρτησης.

Συντακτική Ανάλυση

Η δημιουργία του συντακτικού αναλυτή βασίστηκε στην περσινή γραμματική της γλώσσας CutePy. Υπάρχουν περιπτώσεις που ένας κανόνας απλώς καλεί κάποιον άλλο, εκεί η αντίστοιχη συνάρτηση που δημιουργήθηκε καλεί την συνάρτηση του κανόνα που καλείται.

Στη περίπτωση, όμως, που πρέπει να αναγνωριστεί κάποια ακολουθία συμβόλων τότε καλείται ο λεκτικός αναλυτής ο οποίος επιστρέφει τη «λέξη» (λεκτική μονάδα) που αναγνώρισε, εάν ταυτίζεται με το αναμενόμενο σύμβολο τότε προχωράμε παρακάτω στην αναγνώριση του επόμενου συμβόλου έως ότου αναγνωρίσουμε επιτυχώς όλα τα σύμβολα που αναμένεται να υπάρχουν. Σε περίπτωση που ο λεκτικός αναλυτής δεν επιστρέψει την «λέξη» που αναμένουμε τότε τυπώνεται στην οθόνη το αντίστοιχο μήνυμα σφάλματος και η αντίστοιχη συνάρτηση τερματίζει την λειτουργία της. Η επιστρεφόμενη τιμή του λεκτικού αναλυτή αποθηκεύεται στην μεταβλητή `token`. Στη συνέχεια φαίνονται αναλυτικά οι συναρτήσεις για την λειτουργία του συντακτικού αναλυτή:

- **startRule** : `def_main_part call_main_part`

Με την κλήση της **startRule()** γίνεται η έναρξη του προγράμματος όπου καλούνται κατά σειρά οι συναρτήσεις **def_main_part()** και **call_main_part()**.

- **def_main_part** : `#int' declarations ('def' def_main_function)*`

Στην **def_main_part()** ξεκινάει ο έλεγχος για την ύπαρξη global μεταβλητών που ορίζονται στην αρχή του προγράμματος. Εάν βρεθεί η λεκτική μονάδα **#int** καλείται η συνάρτηση **declarations()**. Στη συνέχεια, όσο παράγεται η λεκτική μονάδα **def** καλείται η συνάρτηση **def_main_function()**.

- **def_main_function** : `'def' funcID '(' temp_id_list ')' ':'
 '#{'
 declarations
 ('def' def_nested_function)*
 check_globals
 statements
 '#}'`

Με την κλήση της **def_main_function()** αναμένεται η δεσμευμένη λέξη **def**, να επακολουθεί το **funcID**, δηλαδή το όνομα της συνάρτησης, το άνοιγμα παρένθεσης **(**, και σε αυτό το σημείο γίνεται ένα έλεγχος για τις προσωρινές μεταβλητές καλώντας την συνάρτηση **temp_id_list(isArg)**. Στη συνέχεια, ακολουθείται το κλείσιμο της παρένθεσης **)**, η άνω κάτω τελεία **:** και το σύμβολο **#{** για την αρχή της συνάρτησης. Έπειτα, αφού έχει ακολουθηθεί επακριβώς η γραμματική της γλώσσας `cry`, καλείται η συνάρτηση **declarations()** και μετά όσο εμφανίζεται δεσμευμένη λέξη **def** καλείται η συνάρτηση **def_nested_function()**. Μετά από αυτόν τον έλεγχο για συναρτήσεις μέσα σε μία συνάρτηση, γίνεται ο έλεγχος για **global** μεταβλητές καλώντας την **check_globals()**. Ακολουθεί η κλήση της συνάρτησης **statements()** και τέλος το σύμβολο για το κλείσιμο μίας συνάρτησης **#}**. Υπάρχουν, επίσης, τα κατάλληλα μηνύματα λάθους που θα τυπωθούν στην οθόνη σε περίπτωση που δεν ακολουθηθεί κάτι σωστά και θα τερματίσει το πρόγραμμα.

- **def_nested_function** : `'def' funcID '(' temp_id_list ')' ':'
 '#{'
 declarations
 ('def' def_nested_function)*
 check_globals
 statements
 '#}'`

Στη συνάρτηση **def_nested_function()** γίνεται ότι ακριβώς και στη **def_main_function()**. Αυτό χρειάζεται για να διαβάζονται οι συναρτήσεις που βρίσκονται φωλιασμένες μέσα σε άλλες.

- **declarations** : ('#int' declaration_line)*

Με την κλήση της **declarations()** ελέγχεται αν η λεκτική μονάδα που έχει παραχθεί είναι η **#int** και όσο ισχύει καλείται η συνάρτηση **declaration_line()**.

- **declaration_line** : id_list

Η συνάρτηση **declaration_line()** καλεί την **id_list(isArg)**.

- **check_globals** : (global_line)*

Με την κλήση της **check_globals(isArg)** ελέγχεται όσο η μεταβλητή **helpGlobal** είναι **false(1)** τότε καλείται η συνάρτηση **global_line(isArg)**.

- **global_line** : id_list

Η συνάρτηση **global_line(isArg)** καλεί την **id_list(isArg)** και αλλάζει την μεταβλητή **helpGlobal** σε **true(0)**.

- **statement** : simple_statement | structured_statement

Η συνάρτηση **statement()** αρχικά ελέγχει αν η λεκτική μονάδα στην οποία βρισκόμαστε ανήκει στα **ID[]**, **temp_ID[]**, **func_ID[]**, ή είναι μία από τις λέξεις 'print' ή 'return' τότε καλεί τη **simple_statement()**. Αλλιώς ελέγχει αν η λεκτική μονάδα στην οποία βρισκόμαστε είναι μία από τις λέξεις 'if', 'elif', 'else' ή 'while' και τότε καλεί τη **structured_statement()**. Σε περίπτωση που δεν ισχύουν οι παραπάνω συνθήκες τυπώνεται στην οθόνη μήνυμα λάθους και τερματίζει το πρόγραμμα.

- **statements** : statement +

Στην **statements()** καλείται η **statement()** και όσο δεν τελειώνει η συνάρτηση, δηλαδή όσο δεν βρίσκουμε το σύμβολο **'#}'** τότε ξανακαλείται η **statement()**.

- **simple_statement** : assignment_stat() | print_stat() | return_stat() | expression()

Κατά την έναρξη της **simple_statement()** ελέγχεται η λεκτική μονάδα που βρισκόμαστε και επιλέγεται ποια συνάρτηση θα κληθεί. Εάν η λεκτική μονάδα ανήκει στο **ID[]** ή στο **temp_ID[]** τότε καλείται η **assignment_stat()**. Εάν η λεκτική μονάδα είναι η λέξη 'print' τότε καλείται η **print_stat()**. Εάν η λεκτική μονάδα είναι η λέξη 'return' τότε καλείται η **return_stat()**. Τέλος, εάν η λεκτική μονάδα ανήκει στο **func_ID[]** τότε καλείται η **expression()**.

- **structured_statement** : if_stat | while_stat

Κατά την έναρξη της **structured_statement()** ελέγχεται η λεκτική μονάδα που βρισκόμαστε και επιλέγεται ποια συνάρτηση θα κληθεί. Εάν η λεκτική μονάδα είναι μία από τις λέξεις 'if', 'elif' ή 'else' τότε καλείται η **if_stat()**. Εάν η λεκτική μονάδα είναι η λέξη 'while' τότε καλείται η **while_stat()**.

- **assignment_stat** : ID/temp_ID
 '=' (expression
 | 'int' '(' 'input' '(' ')'
)

Εφόσον έχει ελεγχθεί στην **simple_statement()** η λεκτική μονάδα ότι ανήκει στο **ID[]** ή στο **temp_ID[]**, στην συνάρτηση **assignment_stat()** αναμένεται η επόμενη λεκτική μονάδα να είναι το σύμβολο '=' (ίσων). Εν συνεχεία αναμένονται δύο δομές. Η πρώτη είναι η λεκτικές μονάδες που ακολουθούν να έχουν τη σειρά: 'int', '(', 'input', '(', ')'. Αλλιώς καλείται η συνάρτηση **expression()**. Τέλος, υπάρχουν και οι αντίστοιχες εκτυπώσεις για τα λάθη και σε αυτή την περίπτωση ο τερματισμός του προγράμματος.

- **print_stat** : 'print' '(' expression ')'

Εφόσον έχει ελεγχθεί στην **simple_statement()** η λεκτική μονάδα 'print', στην συνάρτηση **print_stat()** αναμένεται η επόμενη λεκτική μονάδα να είναι το σύμβολο '(' (άνοιγμα παρένθεσης), έπειτα καλείται η **expression()** και τέλος αναμένεται η λεκτική μονάδα να είναι το σύμβολο ')' (κλείσιμο παρένθεσης). Υπάρχουν οι αντίστοιχες εκτυπώσεις για τα λάθη και σε αυτή την περίπτωση ο τερματισμός του προγράμματος.

- **return_stat** : 'return' expression

Εφόσον έχει ελεγχθεί στην **simple_statement()** η λεκτική μονάδα 'return', στην συνάρτηση **return_stat()** καλείται η συνάρτηση **expression()**.

- **if_stat** : 'if' condition ':' statements
 ('elif' condition ':' statements)*
 ('else' ':' statements)

Η συνάρτηση αυτή υλοποιήθηκε λίγο διαφορετικά καθώς πρώτα ελέγχουμε εάν η λεκτική μονάδα αντιστοιχεί στη λέξη 'else' τότε περιμένουμε το σύμβολο ':' (άνω και κάτω τελεία) και καλείται η **statements()**. Στη συνέχεια εφόσον έχουν ελεγχθεί στην **structured_statement()** οι λεκτικές μονάδες 'if' και 'elif', στην συνάρτηση **if_stat()** καλείται η συνάρτηση **condition()**, περιμένουμε το σύμβολο ':' (άνω και κάτω τελεία) και έπειτα καλείται η συνάρτηση **statements()**. Υπάρχουν οι αντίστοιχες εκτυπώσεις για τα λάθη και σε αυτή την περίπτωση ο τερματισμός του προγράμματος.

- **while_stat** : 'while' condition ':' '#{ statements '#}'

Εφόσον έχει ελεγχθεί στην **structured_statement()** η λεκτική μονάδα 'while', στην συνάρτηση **while_stat()** καλείται η **condition()**, αναμένονται οι επόμενες λεκτικές μονάδες να είναι το σύμβολο ':' (άνω και κάτω τελεία) και το σύμβολο '#{ (με την αντίστοιχη σειρά), έπειτα καλείται η **statements()** και τέλος αναμένεται το σύμβολο '#}'. Υπάρχουν οι αντίστοιχες εκτυπώσεις για τα λάθη και σε αυτή την περίπτωση ο τερματισμός του προγράμματος.

- **id_list** : ID (',' ID)*

Η συνάρτηση **id_list(isArg)** ελέγχει αρχικά αν η λεκτική μονάδα ανήκει στο **ID[]** και έπειτα όσο βρίσκει το σύμβολο ',' (κόμμα) ελέγχει ξανά αν η νέα λεκτική μονάδα ανήκει στο **ID[]**. Αυτό γίνεται μέχρι να σταματήσει να βρίσκει το σύμβολο ',' (κόμμα). Τέλος, υπάρχουν οι αντίστοιχες εκτυπώσεις για τα λάθη και σε αυτή την περίπτωση ο τερματισμός του προγράμματος.

- **temp_id_list** : temp_ID (‘, temp_ID)*

Η συνάρτηση **temp_id_list(isArg)** εκτελείται ακριβώς όπως η **id_list(isArg)** αλλά ελέγχει για το αν η λεκτική μονάδα ανήκει στο **temp_ID[]**.

- **expression** : optional_sign term (ADD_OP term)*

Η συνάρτηση **expression()** ξεκινά καλώντας απευθείας την συνάρτηση **optional_sign()** και στην συνέχεια την **term()**. Μετά την επιστροφή της **term()** ελέγχεται αν η λεκτική μονάδα είναι τα σύμβολα ‘+’ ή ‘-’. Στην συνέχεια καλούνται επαναληπτικά η **ADD_OP()** και η **term()** έως ότου επιστραφεί λεκτική μονάδα διαφορετική από τα σύμβολα ‘+’ και ‘-’.

- **term** : factor (MUL_OP factor)*

Η συνάρτηση **term()** ξεκινά καλώντας απευθείας την συνάρτηση **factor()**. Μετά την επιστροφή της **factor()** ελέγχεται αν η λεκτική μονάδα είναι τα σύμβολα ‘*’, ‘/’ ή ‘%’. Στην συνέχεια καλούνται επαναληπτικά η **MUL_OP()** και η **factor()** έως ότου επιστραφεί λεκτική μονάδα διαφορετική από τα σύμβολα ‘*’, ‘/’ και ‘%’.

- **factor** : INTEGER
 | ID/temp_ID idtail
 | funcID idtail
 | ‘(‘ expression ‘)’
 | ‘int’ assignment_stat

Για την συνάρτηση **factor()** δημιουργήσαμε πέντε αποδεκτές δομές οι οποίες ακολουθούν τη γραμματική της γλώσσας cry. Η πρώτη δομή ελέγχει αν η λεκτική μονάδα στην οποία βρισκόμαστε είναι **ακέραιος αριθμός**. Η δεύτερη δομή ελέγχει αν η λεκτική μονάδα στην οποία βρισκόμαστε είναι τύπου **ID** ή **temp_ID** και αμέσως μετά καλεί την **idtail()**. Η τρίτη δομή είναι ίδια με την δεύτερη απλά ελέγχει για τύπου **func_ID**. Η τέταρτη δομή ξεκινά ελέγχοντας για την ύπαρξη του συμβόλου ‘(‘, καλεί την **expression()** και περιμένει ξανά το σύμβολο ‘)’. Τέλος, ακολουθώντας την πέμπτη δομή ελέγχουμε αν η λεκτική μονάδα στην οποία βρισκόμαστε είναι η λέξη **‘int’** και έπειτα καλούμε την **assignment_stat()**.

- **idtail** : ‘(‘ actual_par_list ‘)’ | ε

Η συνάρτηση **idtail(fid)** ελέγχει αρχικά αν η λεκτική μονάδα είναι το σύμβολο ‘(‘. Αν ισχύει τότε καλεί την **actual_par_list()** και μετά περιμένει το κλείσιμο της παρένθεσης ‘)’. Η συνάρτηση αυτή επιστρέφει το όρισμά της (fid).

- **actual_part_list** : expression (‘, expression)*

Η συνάρτηση **actual_part_list()** καλεί απευθείας την **expression()**. Κατά την επιστροφή της ελέγχεται επαναληπτικά αν η λεκτική μονάδα είναι το σύμβολο ‘,’ (κόμμα) και όσο το βρίσκει καλεί ξανά την **expression()**.

- **condition** : bool_term ('or' bool_term)*

Η συνάρτησή **condition()** καλεί απευθείας την **bool_term()**. Κατά την επιστροφή της ελέγχει επαναληπτικά αν η λεκτική μονάδα είναι η λέξη **‘or’** και καλεί την **bool_term()** εκ νέου. Η επαναληπτική διαδικασία σταματάει όταν δεν βρίσκει τη λέξη **‘or’**.

- **bool_term** : bool_factor ('and' bool_factor)*

Η συνάρτησή **bool_term()** καλεί απευθείας την **bool_factor()**. Κατά την επιστροφή της ελέγχει επαναληπτικά αν η λεκτική μονάδα είναι η λέξη '**and**' και καλεί την **bool_factor()** εκ νέου. Η επαναληπτική διαδικασία σταματάει όταν δεν βρίσκει τη λέξη '**and**'.

- **bool_factor** : 'not' condition ':' | expression REL_OP expression

Για την συνάρτηση **bool_factor()** αρχικά ελέγχεται αν η λεκτική μονάδα στην οποία βρισκόμαστε είναι η λέξη '**not**'. Καλείται η **condition()** και μετά περιμένει ':' (άνω και κάτω τελεία). Αλλιώς, εάν δεν βρεθεί '**not**' τότε καλούνται με σειρά οι **expression()**, **REL_OP()** και πάλι η **expression()**.

- **optional_sign** : ADD_OP | ε

Στην **optional_sign()** καλείται μόνο η συνάρτηση **ADD_OP()**.

- **call_main_part** : 'main' main_function_call

Με την κλήση της **call_main_part()** αναμένεται η δεσμευμένη λέξη **main** και στη συνέχεια καλείται η συνάρτηση **main_function_call()**. Εάν δεν βρεθεί η λέξη **main** τότε εμφανίζεται μήνυμα λάθους και τερματίζεται το πρόγραμμα.

- **main_function_call** : declarations (statement)* until EOFError

Η συνάρτηση **main_function_call()** καλεί την **declarations()**. Έπειτα καλείται επανειλημμένα η **statement()** μέχρι να προκαλέσει εξαίρεση '**EOFError**' (που υποδηλώνει το τέλος του αρχείου), ο βρόχος τερματίζει και η συνάρτηση **main_function_call()** επιστρέφει.

- **ADD_OP** : '+' | '-'

Κατά την κλήση της **ADD_OP()** ελέγχεται αν η λεκτική μονάδα αντιστοιχεί σε κάποιο από τα σύμβολα '+', '-' και τα επιστρέφει αντίστοιχα.

- **MUL_OP** : '*' | '/' | '%'

Κατά την κλήση της **MUL_OP()** ελέγχεται αν η λεκτική μονάδα αντιστοιχεί σε κάποιο από τα σύμβολα '*', '/', '%' και τα επιστρέφει αντίστοιχα.

- **REL_OP** : '==' | '<=' | '>=' | '>' | '<' | '!='

Κατά την κλήση της **REL_OP()** ελέγχεται αν η λεκτική μονάδα αντιστοιχεί σε κάποιο από τα σύμβολα '==', '<=', '>=', '>', '<', '!=' και τα επιστρέφει αντίστοιχα.

Κατά την ολοκλήρωση της λειτουργίας του συντακτικού αναλυτή εμφανίζεται το μήνυμα «Syntax Analysis Successful!».

Για τον έλεγχο της πρώτης φάσης δημιουργήθηκαν δύο cpy προγράμματα.

test1.cpy : Αυτό το μικρό πρόγραμμα φτιάχτηκε για να ελέγξουμε ότι δουλεύουν σωστά η δομή if-else και οι φωλιασμένες συναρτήσεις

```
def main_ex():
#{
    #int b,c,g
    def P1(x,y):
        #{
            y=y-1
            if x==1:
                return x
            else:
                return P1((x-1),y)
        #}
    c=10
    b=5
    g=P1(c, b)
#}

#def main
## call of main functions ##
main_ex()
```

test2.cpy : Σε αυτό το πρόγραμμα δοκιμάσαμε και κάποιες πιο σύνθετες μαθηματικές ακολουθίες για να διαπιστώσουμε ότι αναγνωρίζονται σωστά.

```
def main_quadratic():
#{

    #int a, b, c, d

    def square(x):
        #{
            return x * x
        #}

    def root(x):
        #{
            #int i
            i = 1
            while i*i < x:
                #{
                    i = i + 1
                    return i
                #}
            #}
#}
```



```

a = int(input())
b = int(input())
c = int(input())

d = square(b) - 4 * a * c

print(square(b))
print(root(a))
#}

#def main
main_quadratic()

```

Φάση 2

Παραγωγή Ενδιάμεσου Κώδικα

Με στόχο την παραγωγή τετράδων (quad) αρχικά δημιουργήσαμε την λίστα quadList, στην οποία γίνεται αποθήκευση των quad (= [cnt, operation, x, y, z]) η οποία συνδυάζεται με το quadCounter(= αριθμητή-ετικέτα του επόμενου quad).

Υλοποίηση των συναρτήσεων:

- **nextQuad()** η οποία επιστρέφει την ετικέτα της επόμενης τετράδας.
- **genQuad(op, x, y, z)** η οποία παράγει την τετράδα από τα ορίσματα που δέχεται και τα προσθέτει στη λίστα.
- **newTemp()** είναι συνάρτηση υπεύθυνη για την δημιουργία προσωρινών μεταβλητών που χρησιμεύουν για την αποθήκευση αριθμητικών ή/και λογικών πράξεων αλλά και αποθήκευση επιστρεφόμενων τιμών από κάλεσμα συναρτήσεων. Για την λειτουργικότητα της συνάρτησης προστέθηκε λίστα tempList που αποθηκεύονται οι μεταβλητές “T_tempCounter ” όπου tempCounter = 1,2,3...
- **emptyList()** απλώς επιστρέφει μια κενή λίστα
- **makeList(x)** επιστρέφει list με το x μοναδικό όρισμα.
- **merge(list1, list2)** ενώνει τις λίστες list1 κ’ list2
- **backPatch(lista, k)** η οποία χρησιμεύει να συμπληρώσει το 4^ο όρισμα του quad, το οποίο περνάει ως lista, με την τιμή k. Σε μελλοντικό χρόνο αυτή η τιμή, εφόσον η συνάρτηση καλείτε ορθώς, είναι το cnt του quad που θα μεταπηδήσει το παρόν quad.
- **intFile(file)** απλώς δημιουργεί αρχείο όπου καταγράφονται τα quads.

Προσθήκη των συναρτήσεων στον συντακτικό αναλυτή

Ξεκινάμε δημιουργώντας με την **genQuad()** για κάθε συνάρτηση που εντοπίζεται. Μέχρι να βγούμε από οποιαδήποτε συνάρτηση, στην **statements()** και τις εμφωλιασμένες συναρτήσεις της θα συναντήσουμε τις υπόλοιπες δημιουργίες τετράδων. Πιο συγκεκριμένα συναντάτε στις:

- ✓ **simple_statement()**
- ✓ **assignment_stat()**
- ✓ **print_stat()**
- ✓ **return_stat()**
- ✓ **if_stat()**
- ✓ **while_stat()**
- ✓ **expression()**
- ✓ **term()**
- ✓ **idtail()**
- ✓ **actual_par_list()**
- ✓ **bool_factor()**

Στο τέλος κάθε συνάρτησης κλείνουμε με δημιουργία quad για το τέλος της εκάστοτε συνάρτησης που διάβασε ο αναλυτής. Μια ιδιαιτερότητα εντοπίζεται στο τέλος της main όπου απαιτείται ένα ιδικό quad(= "halt", "_", "_", "_").

Τα ορίσματα ενός quad διαφέρουν σύμφωνα με τη συνάρτηση στην οποία δημιουργείται. Όσο αναφορά την θέση στον κώδικα ακολουθούμε τις ενδείξεις της θεωρίας. Για την πλήρη λειτουργικότητα απαιτείται ο τέλειος συνδυασμός με τις υπόλοιπες συναρτήσεις. Με τις πιο ειδικές συναρτήσεις να είναι η **newTemp()** και **backpatch()** σημειώνουμε τις συναρτήσεις που εντοπίζουμε την κάθε μία.

newTemp()

expression()

term()

idtail()

backpatch():

if_stat()

while_stat()

condition()

bool_term()

Αξίζει να δοθεί ένα παράδειγμα εκτέλεσης εντολή προς εντολή.

Έστω ο κώδικας:

```
while i<=2000:
#{
    print(leap(i))
    i = i + 400
#}
```

statement() → **structured_statement()** → **while_stat()** {γίνεται nextQuad() αφού βρεθήκαμε στη while} → **condition()** → **bool_term()** → **bool_factor()** όπου θα δημιουργηθεί η επιστρεφόμενη λίστα Rtrue = makeList(nextQuad) η οποία δείχνει στο

πρώτο quad (= (<=, i, 2000, “_”) σε περίπτωση που η συνθήκη συνεχίζει να ισχύει και επιστρεφόμενη λίστα Rfalse = makeList(nextQuad) η οποία δείχνει στο quad άλματος (“jump”, “_”, “_”, “_”) για την έξοδο από το while.

Ακολουθεί αντίστροφο κάλεσμα το συναρτήσεων με τις επιστρεφόμενες τιμές να περνάν σε τοπικές μεταβλητές και να επιστρέφονται ανάλογα.

bool_factor() {returns boolTrue = Rtrue, boolFalse = Rfalse} →

condition() {returns condTrue = boolTrue, condFalse = boolFalse} →

while_stat() { **backpatch(condTrue, nextQuad())** το οποίο είναι το quad αφού περαστεί η συνθήκη και άρα αποθηκεύει τη θέση του, μέχρι ο λεκτικός να διαβάσει έξω από το while όπου θα πρέπει να υπάρξει quad για jump πριν το while άρα και backpatch για την περίπτωση αυτή}.

Μέσα στη **statement()** του σώματος της while για να παρατηρήσουμε την **newTemp()** σε λειτουργία, ο λεκτικός πρέπει να φτάσει μέσω της expression να βρει token == ‘+’ στην γραμμή i = i + 400 και να αποθηκεύσει σε μια “T_” την τιμή (i+400) και ύστερα την “T_” στην i.

Πίνακας Συμβόλων

Για τον πίνακα συμβόλων δημιουργήσαμε μια class Entity η οποία δημιουργεί το entity αντικείμενο που χρειαζόμαστε ανάλογα και η μέθοδος που θα χρησιμοποιηθεί (var, function, param, Temp). Δημιουργούμε επίσης απαραίτητες λίστες όπως scopes, scope για την αποθήκευση score μέσα στην scopes, την varID για αποθήκη των variables, την argList για αποθήκη των argument των συναρτήσεων και την helpList που δρα σαν βοηθητική για την επεξεργασία των scopes την στιγμή που θα χρειαστεί να γίνει παραγωγή του τελικού κώδικα.

Επιτρεπόμενα Entity αντικείμενα:

```
@classmethod
def var(cls, id, state, offset):
    variable = cls(id, state)
    variable.offset = offset
    return variable

@classmethod
def function(cls, id, state):
    func = cls(id, state)
    func.squad = 0
    func.args = []
    func.length = 0
    return func

@classmethod
def param(cls, id, state, mode, offset):
    parameter = cls(id, state)
    parameter.mode = mode
    parameter.offset = offset
    return parameter
```

```
@classmethod
def Temp(cls, id, state, offset):
    temp = cls(id, state)
    temp.offset = offset
    return temp
```

Υλοποίηση των συναρτήσεων:

- **newScope()** φτιάχνει ένα scope = [id, len(scopes), [] -> δυνητικά θα αποθηκεύεται κάποιο Entity]
- **remScope()** αφαιρεί το τελευταίο scope από την scopes
- **newEntity(entity)** προσθέτει ένα entity στο 3^ο όρισμα του scope και από το τέλος της λίστας scopes
- **addArgs(entity)** προσθέτει στο entity function στο πεδίο args την argList, δηλαδή τα arguments της συνάρτησης που έχουν αποθηκευτεί.
- **getOffset()** επιστρέφει την τιμή του offset αν πρόκειται entity που μας ενδιαφέρει το πεδίο του.

Δημιουργία Entity κατά την λεκτική και συντακτική ανάλυση

Η δημιουργία του Entity ξεχωρίζει με βάση τη μορφή του συμβολικού ονόματος που θέλουμε να αποθηκεύσουμε την πληροφορία του, κάτι που θα αξιοποιηθεί για την τελική παραγωγή του κώδικα. Η λογική της αποθήκευσης ενός Entity είναι: αρχικά η δημιουργία ενός scope όπου εντοπίζεται συνάρτηση με **newScope(func_name)** και έπειτα δημιουργία και αποθήκευση του entity. Όταν θα διαβαστούν μεταβλητές, παράμετροι συναρτήσεων ή προσωρινές μεταβλητές, τότε επίσης ακολουθούμε απλά δημιουργία entity και αποθήκευση. Οι συναρτήσεις που εντοπίζεται η λειτουργικότητα είναι:

- ✓ **def_main_function()**
- ✓ **def_nested_function()**
- ✓ **id_list()** και **temp_id_list()**
- ✓ **id_tail()**
- ✓ **actual_par_list()**
- ✓ **main_function_call()**
- ✓ **newTemp()**

Για τον έλεγχο της ορθότητας χρησιμοποιούμε την **outputSymbolFile()**.

Στο symbol_test2.cpy:

```
#int x
def p1(x,y):
#{
    #int a,b,c

    def f1(x):
    #{
        #int a
        b = a
        a = x
        c = f1(x)
        return c
    #}
#}
```

```

def f2(x):
    #{
        c = f1(x)
        return c
    #}
    y = x
#}

#def main
print(f1(1))
print(f2(2))

```

Παρατηρούμε πως ξεκινάμε από την p1 πηγαίνουμε στη f1, μετά αντίστοιχα στη p1 η f2 ενώ στο τέλος μένει μόνο η p1 αφού ο ορισμός των f1 και f2 έχει καταγραφεί και έχει αφαιρεθεί από τη λίστα scopes.

```

=====
Scope 1
Function entity: f1, starting quad: 9, length: 28
Parameter entity: x, mode: cv, offset: 12
Variable entity: a, offset: 16
Parameter entity: x, mode: CV, offset: 20
Temporary variable entity: T_1, offset: 24

Scope 0
Function entity: p1, starting quad: 0, length: 0
Parameter entity: x, mode: cv, offset: 12
Parameter entity: y, mode: cv, offset: 16
Variable entity: a, offset: 20
Variable entity: b, offset: 24
Variable entity: c, offset: 28

=====
Scope 1
Function entity: f2, starting quad: 16, length: 24
Parameter entity: x, mode: cv, offset: 12
Parameter entity: x, mode: CV, offset: 16
Temporary variable entity: T_2, offset: 20

Scope 0
Function entity: p1, starting quad: 0, length: 0
Parameter entity: x, mode: cv, offset: 12
Parameter entity: y, mode: cv, offset: 16
Variable entity: a, offset: 20
Variable entity: b, offset: 24
Variable entity: c, offset: 28

=====
Scope 0
Function entity: p1, starting quad: 19, length: 32
Parameter entity: x, mode: cv, offset: 12
Parameter entity: y, mode: cv, offset: 16
Variable entity: a, offset: 20
Variable entity: b, offset: 24
Variable entity: c, offset: 28

```

Φάση 3

Τελικός Κώδικας

Για την ολοκλήρωση του κώδικα δημιουργήθηκαν βοηθητικές συναρτήσεις που μας βοηθούν στην υλοποίηση των βασικών για την παραγωγή.

Βοηθητικές:

- **search_scope_id(id)** την υλοποιήσαμε για να επιστρέψουμε αν υπάρχει το entity με το id που μας ενδιαφέρει από την helpList.
- **count_function_occurrences()** το σκεπτικό υλοποίησης είναι να κρατάει με ένα dictionary για την εκάστοτε συνάρτηση τον αριθμό εμφανίσεων τους στο πρόγραμμα.
- **generateMainList()** δημιουργήθηκε για να σιγουρευτούμε ότι βρίσκει την main σαν entity και την αποθηκεύουμε μοναδικά στην mainList.
- **outputSymbolFile()** ανοίγει αρχείο για να κάνει print τις πληροφορίες που μας ενδιαφέρουν για το entity.
- **remHelp(entityID)** διαγράφει από την helpList, που πλέον κρατάει τα scope, το entity με το entityID.
- **generateHelpList()** απλώς δημιουργεί την λίστα helpList κενή.

Βασικές:

- **gnvcode(v)** η οποία παράγει τελικό κώδικα ώστε να γίνεται προσπέλαση των μεταβλητών ή διευθύνσεων οι οποίες ανήκουν σε διαφορετικό εγγράφημα δραστηριοποίησης από αυτό της συνάρτησης που αυτή τη στιγμή μεταφράζεται.
- **loadvr(v,reg)** η οποία καταγράφει για τον καταχωρητή reg το offset της μεταβλητής, παραμέτρου ή προσωρινής μεταβλητής v ώστε ο τελικός κώδικας να περιλαμβάνει τις εντολές load με κατάλληλες διευθύνσεις.
 - αν v είναι αριθμός: li reg,v
 - αν v είναι global μεταβλητή δηλαδή ανήκει στο κυρίως πρόγραμμα: lw reg,-offset(\$s0)
 - αν η v έχει δηλωθεί στη συνάρτηση που αυτή τη στιγμή εκτελείται και είναι τοπική μεταβλητή, ή τυπική παράμετρος που περνάει με τιμή, ή προσωρινή μεταβλητή: lw reg,-offset(\$sp)
 - αν η v έχει δηλωθεί σε κάποιο πρόγονο και εκεί είναι τοπική μεταβλητή, ή τυπική παράμετρος που περνάει με τιμή: gnlvcode() lw \$reg,(\$t0)
- **storerv(reg,v)** η οποία καταγράφει για τον καταχωρητή reg το offset της μεταβλητής, παραμέτρου ή προσωρινής μεταβλητής v ώστε ο τελικός κώδικας να κάνει store τις κατάλληλες διευθύνσεις.
 - αν v είναι αριθμός: li reg,v
 - αν v είναι global μεταβλητή, προσωρινή(τοπική) μεταβλητή ή παράμετρος και ανήκει στο κυρίως πρόγραμμα: sw reg,-offset(\$sp)
 - αν η v έχει δηλωθεί σε κάποιο πρόγονο και εκεί είναι τοπική μεταβλητή, ή τυπική παράμετρος που περνάει με τιμή: gnlvcode() lw \$reg,(\$t0)

- **produce(file)** τελική συνάρτηση που τρέχει όλα τα quads και συμπεριφέρεται αναλόγως το id τού. Καταγράφει τις εντολές RISC-V σε ένα αρχείο .asm.

Τέλος, θα θέλαμε να αναφέρουμε ότι για τον έλεγχο και των τριών φάσεων της εργασίας χρησιμοποιούσαμε το αρχείο test.cpy που είχε δοθεί αρχικά.